



PROJECT REPORT OF CO-OP

Project at Industry (Module-2)

ON

**Phishing Detection Using Machine Learning:
Evaluating Classical Classifiers for Web-Based Phishing Detection**

submitted in partial fulfilment of the requirements for the award of degree of

BACHELOR OF ENGINEERING

In

COMPUTER SCIENCE AND ENGINEERING

Submitted by:

Keshav Gupta (2210990505)
Muskan Saini (2210991945)
Aishwary Chauhan (2210991209)
Tejasvi Jhamta (2210992464)

Supervised By:

Darpan Anand
Dean of CSE
Chitkara University

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
CHITKARA UNIVERSITY INSTITUTE OF ENGINEERING AND TECHNOLOGY
CHITKARA UNIVERSITY, PUNJAB, INDIA**

CONTENTS

S.No.	Title	Page No.
	Declaration	iii
	Acknowledgement	iv
	List of Figures and Tables	v
	Abstract	vi
1	Introduction	1
2	Methodology	3
3	Tools and Technologies	5
4	Implementation	7
5	Major Findings / Outcomes / Results	9
6	Conclusion and Future Scope	11
	References	12
	Appendices	13

DECLARATION

I hereby certify that the work which is being presented in the project report entitled "**Phishing Detection Using Machine Learning: Evaluating Classical Classifiers for Web-Based Phishing Detection**" in partial fulfilment of requirement for the award of the degree of Bachelor of Engineering (Computer Science and Engineering) submitted in the department of Computer Science and Engineering at Chitkara University Institute of Engineering and Technology, Chitkara University, Punjab, India, is an authentic record of my own work carried out under the supervision of <Name of the Mentor>. The matter presented in this project report has not been submitted in any other university/institute for the award of any degree.

Place: Rajpura

Date: 18 May 2026

This is to certify that the above statement made by the candidate is correct to the best of my knowledge and belief.

Darpan Anand

Dean of CSE

Department of Computer Science and Engineering

Chitkara University Institute of Engineering and Technology,

Chitkara University, Punjab, India

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our mentor and supervisor for providing invaluable guidance and support throughout the course of this project. We are thankful to the Department of Computer Science and Engineering, Chitkara University Institute of Engineering and Technology, for providing us the opportunity and resources to carry out this research work.

We extend our thanks to the open-source community behind scikit-learn, Flask, and the Kaggle dataset contributors, without whom this project would not have been possible. Special thanks to our families and friends for their constant encouragement.

Finally, we acknowledge the contributions of all researchers and authors whose prior work in the domain of phishing detection and machine learning formed the foundation for this study.

Keshav Gupta

Muskan Saini

Aishwary ChauhanTejasvi Jhamta

LIST OF FIGURES AND TABLES

List of Figures

Figure No.	Caption	Page No.
Fig. 1	Phishing growth, sector targets, and financial losses (2019–2024)	2
Fig. 2	End-to-end system pipeline	4
Fig. 3	Training vs. Validation Accuracy LR and MNB	9
Fig. 4	Performance comparison across all metrics	10
Fig. 5	Confusion matrices LR (left) and MNB (right)	10
Fig. 6	ROC curves with AUC values	11

List of Tables

Table No.	Caption	Page No.
Table I	Summary of Related Work	3
Table II	Dataset Statistics	4
Table III	Experimental Configuration	5
Table IV	Performance Metrics Comparison	9

ABSTRACT

Phishing continues to be a major challenge in the cybersecurity landscape. This project explores how far classical, lightweight machine learning models can go against this problem without demanding heavy computational resources. Working on a 549,347-sample Kaggle dataset of labelled URLs, a TF-IDF-based pipeline was constructed and two classifiers were evaluated: Logistic Regression (LR) as the primary model and Multinomial Naive Bayes (MNB) as a baseline comparison.

Neither model loads the actual webpage classification is performed entirely on the raw URL string, with no WHOIS queries, no page rendering, and no outbound network calls. Logistic Regression achieved 96.4% accuracy with an AUC of 0.984, while Multinomial Naive Bayes reached 94.1% accuracy with an AUC of 0.971. Both classifiers operated in sub-millisecond inference time per URL and were deployed through a Flask web application for live testing.

Keywords: Phishing Detection, URL Classification, Logistic Regression, Naive Bayes, TF-IDF, Machine Learning, Cybersecurity, Flask.

1. INTRODUCTION

1.1 Background

Phishing has been around for decades and has grown more dangerous rather than less. The core attack technique has not shifted significantly duping someone into clicking a spoofed link and surrendering credentials or payment details but the volume has grown dramatically. APWG data shows that the number of detected phishing sites jumped from roughly 266,000 in 2019 to well over 1.6 million by 2024. Financial damage tracked a similar trajectory, climbing from around USD 1.8 billion in 2019 to an estimated USD 24.7 billion by 2024.

1.2 Problem Statement

The standard countermeasure to phishing is a blocklist a catalogue of known malicious URLs from which access is denied. The flaw is obvious: blocklists are perpetually reactive. By the time a phishing domain makes it onto the list it has usually already served its purpose. Machine learning flips this around: rather than waiting for known threats to be catalogued, the model learns the structural signatures of phishing URLs so it can flag novel ones it has never encountered before. Restricting classification to URL string features alone no page loads, no WHOIS lookups keeps inference times in the millisecond range with zero external dependencies.

1.3 Objectives

This project describes the construction of a lightweight detection pipeline, specifically:

- Constructing a reproducible TF-IDF vectorisation pipeline operating directly on raw URL strings.
- Running a controlled comparison of Logistic Regression against Multinomial Naive Bayes on 549,347 labelled examples.
- Packaging the best model into a Flask web application for real-time use.
- Evaluating results thoroughly using accuracy, F1, AUC-ROC, and confusion matrices.

1.4 Phishing Types

Phishing is fundamentally a social engineering attack where the attacker poses a legitimate entity to coax the target into handing over sensitive information. Rather than exploiting software vulnerabilities, it exploits human psychology urgency cues, perceived authority, and visual mimicry. Common types include:

- Email Phishing: Bulk email campaigns pushing recipients toward fake websites.
- Spear Phishing: Personalised attacks aimed at specific people or organisations.
- Smishing/Vishing: SMS or voice-based credential theft.
- Web/URL Phishing: Fake sites with deceptive URLs the focus of this study.
- Pharming: DNS/hosts file manipulation to silently redirect users.

Phishing URLs are typically longer than legitimate ones, laced with deceptive keywords (e.g., 'secure-login', 'paypal-verify'), use hyphens and random alphanumeric tokens, and employ non-standard TLDs. It is precisely these lexical signals that the feature extraction pipeline targets.

2. METHODOLOGY

2.1 Related Work

ML-driven approaches to phishing detection have been an active area of research for roughly two decades. The highest-accuracy systems generally depend on hand-crafted features URL length, dot counts, WHOIS-derived domain age or full DOM parsing. The approach in this project bypasses all of that and feeds raw URL strings straight into a TF-IDF vectoriser, simplifying deployment and keeping inference fast.

Table I: Summary of Related Work

#	Authors	Method	Accuracy
1	Ma et al. [1]	LR + lexical + WHOIS; online	~99%
2	Garera et al. [2]	LR on URL structure	~95%
3	Mohammad et al. [3]	Decision Tree; 30 features	97.9%
4	Sahingo et al. [4]	7 ML models; 73 URL features	97.98%
5	Zouina & Outtaj [5]	NB on PhishTank features	93.5%
6	Vazhayil et al. [6]	LR, SVM, RF, KNN	96.3%
7	Yuan et al. [7]	CNN char-level URL	98.2%
8	Le et al. [8]	URLNet: char + word CNN	98.05%
9	Feng et al. [9]	Graph-based URL networks	~97.1%
10	Rao & Pais [10]	DOM + URL; Chrome extension	~98.4%

2.2 Dataset

The Phishing Site URLs dataset from Kaggle was used, aggregating labelled URLs from PhishTank, DMOZ, OpenPhish, and ISCX-URL 2016, providing a reasonably diverse mix of legitimate and phishing URLs.

Table II: Dataset Statistics

Attribute	Details
Total Samples	549,347 URLs
Legitimate	428,103 (77.93%)
Phishing	121,244 (22.07%)
Feature	Raw URL string
Label	good / bad (binary)
Train / Test Split	80% / 20% (seed = 42)

The dataset is significantly imbalanced roughly 78% of URLs are benign with phishing URLs making up just 22%. Recall and F1 are therefore emphasised over raw accuracy, given that a missed phishing URL typically has worse consequences than a false positive.

2.3 Feature Extraction

Instead of manual feature engineering, raw URL strings are fed into a TF-IDF vectoriser set to extract character n-grams (3 to 5 characters wide, char_wb analyser, up to 500K features). The vectoriser surfaces recurring character patterns such as 'paypal-secure' or 'login-verify' that appear disproportionately in phishing URLs without any explicit feature specification.

3. TOOLS AND TECHNOLOGIES

3.1 Programming Environment

The entire pipeline was developed in Python 3.10. Machine learning components vectorisation, model training, evaluation, and serialisation were implemented using scikit-learn 1.3. The web application interface was built with Flask 3.0, and models were serialised using Python's built-in pickle module.

Table III: Experimental Configuration

Parameter	Value
Language	Python 3.10
ML Library	scikit-learn 1.3
Framework	Flask 3.0
Vectoriser	TF-IDF (char_wb, 3–5g, 500K features)
Train / Test Split	80% / 20% (seed = 42)
Evaluation Metrics	Accuracy, Precision, Recall, F1, AUC
Hardware	Intel Core i5, 8 GB RAM
Serialisation	Python pickle (.pkl)

3.2 Machine Learning Models

3.2.1 Why Classical ML?

Classical ML was chosen over deep learning for practical reasons: training completes in seconds on a standard CPU with no GPU required, inference is essentially instantaneous, and both models are transparent. With Logistic Regression the learned weight vector shows exactly which URL substrings push the score toward phishing crucial for security analyst trust.

3.2.2 Logistic Regression (Primary Model)

Logistic Regression models the probability that a URL belongs to the phishing class: $P(y=1|x) = \sigma(w^T x + b)$. L2 regularisation was applied, with penalty strength C selected by cross-validation to

prevent overfitting on the large sparse TF-IDF feature space. LR's well-calibrated probability outputs allow the detection threshold to be tuned to favour recall or precision depending on the deployment scenario.

3.2.3 Multinomial Naive Bayes (Baseline)

Multinomial Naive Bayes operates under the naive conditional independence assumption: $P(y|x) \propto P(y) \times \prod_i P(x_i|y)$. Laplace smoothing was applied with alpha tuned by cross-validation. MNB has a closed-form solution so it converges approximately 3–4× faster than LR. Despite the simplifying independence assumption, MNB is a reliably strong baseline when the input is token frequency data such as TF-IDF vectors.

4. IMPLEMENTATION

4.1 System Pipeline

The end-to-end pipeline proceeds as follows: a user submits a URL via the Flask interface; the backend strips the protocol prefix and sends the cleaned string through the pre-trained vectoriser, producing a sparse feature vector; that vector is passed to the active classifier which returns a phishing probability score; scores at or above 0.5 are classified as phishing, anything below is treated as legitimate.

4.2 Pipeline Steps

- Raw URL input received via Flask web form.
- Preprocessing: strip http(s):// and www. prefixes via regex.
- TF-IDF Vectorization: Cleaned URL transformed by pre-fitted vectorizer.pkl.
- Model Inference: LR or MNB outputs a probability score for the phishing class.
- Threshold & Label: score $\geq 0.5 \rightarrow$ Phishing; else Legitimate.
- Result: "Safe  " or "Phishing  " displayed in real time.

4.3 Model Training

Both models were trained on 80% of the dataset (approximately 439,477 URLs) and tested on the remaining 20% (approximately 109,870 URLs). The TF-IDF vectoriser was fitted only on the training set to avoid data leakage. Cross-validation was used to select hyperparameters for both classifiers.

4.4 Deployment

Both models were serialised using Python's pickle module. At startup, the Flask application loads phishing.pkl (the LR model), with phishing_mnb.pkl available as an alternate. The same preprocessing regex and vectorizer.pkl are used at inference time to guarantee feature-space consistency across training and production environments.

5. MAJOR FINDINGS / OUTCOMES / RESULTS

5.1 Performance Summary

Table IV presents the full performance comparison between Logistic Regression and Multinomial Naive Bayes across all evaluation metrics.

Table IV: Performance Metrics Comparison

Metric	Logistic Regression	Naive Bayes
Accuracy	96.4%	94.1%
Precision	95.8%	93.5%
Recall	94.2%	91.8%
F1-Score	95.0%	92.6%
AUC-ROC	0.984	0.971
Training Time	~12 seconds	~4 seconds
Inference Time	< 1 ms	< 1 ms

5.2 Learning Curve Analysis

LR's training accuracy reached 96.4%, with validation closely following at 95.1% the narrow-spread rules out significant overfitting. MNB converged more quickly but plateaued at lower values: 93.5% training and 91.2% validation. Neither model exhibits serious overfitting behaviour.

5.3 Cross-Metric Analysis

LR outperforms MNB across every metric. Recall shows the starkest gap: 94.2% versus 91.8%. That 2.4-point margin translates into substantially fewer missed phishing URLs. In practice, each missed phishing site is a potential compromise, so that margin is operationally meaningful.

5.4 Confusion Matrix Analysis

On the held-out test set of approximately 109,870 URLs, LR caught 15,948 phishing URLs while letting 5,622 slips through a false negative rate of roughly 26%. MNB fared worse, missing 7,984 phishing URLs with a 37% miss rate. False positives sat at 4,379 for LR versus 6,767 for MNB. In real deployment, the false negative gap is what matters most.

5.5 ROC Curve Analysis

LR's AUC of 0.984 reflects very strong class separability. MNB's AUC of 0.971 is also solid and far above chance. Both curves climb steeply in the low false-positive rate region, meaning a high-precision operating point can still maintain strong recall.

5.6 Advantages Over Existing Work

- No page access needed: classification done purely from the URL string with no network requests, no browser rendering, and no WHOIS calls.
- Real-time deployment: the Flask app classifies URLs in under 1 ms a figure rarely reported in academic work.
- Interpretable: Inspecting LR's weight vector reveals exactly which URL substrings drive phishing scores.
- Scalable: Full retrain completes in ~12 seconds on commodity hardware, enabling daily or weekly refresh without GPU.
- Competitive: 96.4% accuracy with no manually crafted features, vs. 97.98% for Sahingoz et al. who rely on 73 engineered features.

6. CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

This project addressed a concrete question: is it possible to build a fast, practical phishing detector that operates on URL strings alone using only classical machine learning? The experiments confirm the answer is largely yes. Logistic Regression achieved 96.4% accuracy, 95.0% F1, and AUC 0.984 on 549,347 labelled URLs, clearly outperforming MNB (94.1%, 92.6% F1, AUC 0.971) on all reported metrics, with both models staying under 1 ms per URL and requiring no GPU.

More broadly, this work illustrates that deep learning is not a prerequisite for near-state-of-the-art phishing detection. A carefully tuned TF-IDF + Logistic Regression pipeline covers most of the performance gap at a fraction of the complexity. The resulting classifier is deployable directly in browser extensions, DNS layer filters, or API gateways wherever URL-level screening is warranted.

6.2 Limitations

The clearest weakness is against zero-day campaigns using freshly registered, clean-looking short domains nothing in the URL string signals danger in such cases. The model must also be periodically retrained as attackers adapt their URL crafting tactics over time.

6.3 Future Scope

- Incorporating lightweight DOM metadata without requiring full browser rendering to improve detection of zero-day campaigns.
- Constructing an ensemble combining Logistic Regression, MNB, and Random Forest to improve robustness.
- Adopting incremental online learning to enable continuous model adaptation as new phishing patterns emerge.

- Packaging the classifier as a browser extension for tab-level URL screening in real-user browsing environments.

REFERENCES

- [1] J. Ma, L. K. Saul, S. Savage, and G. M. Voelker, "Identifying suspicious URLs: An application of large-scale online learning," in Proc. ICML, 2009, pp. 681–688.
- [2] S. Garera, N. Provos, M. Chew, and A. D. Rubin, "A framework for detection and measurement of phishing attacks," in Proc. ACM WORM, 2007, pp. 1–8.
- [3] R. M. Mohammad, F. Thabtah, and L. McCluskey, "Predicting phishing websites based on self-structuring neural network," *Neural Comput. Appl.*, vol. 25, no. 2, pp. 443–458, 2014.
- [4] O. K. Sahingoz, E. Buber, O. Demir, and B. Dirir, "Machine learning based phishing detection from URLs," *Expert Syst. Appl.*, vol. 117, pp. 345–357, 2019.
- [5] M. Zouina and B. Outtaj, "A novel lightweight URL phishing detection system using SVM and similarity index," *Human-centric Comput. Inf. Sci.*, vol. 7, no. 1, pp. 1–13, 2017.
- [6] A. Vazhayil, R. Vinayakumar, and K. P. Soman, "Comparative study of ML-based approach for URL-based phishing detection," in Proc. ICSCC, 2018.
- [7] J. Yuan, G. Chen, S. Bai, and Y. Duan, "A novel approach for malicious URL detection based on character-level CNN," in Proc. IEEE ICDEW, 2018.
- [8] H. Le, Q. Pham, D. Sahoo, and S. Hoi, "URLNet: Learning a URL representation with deep learning for malicious URL detection," *arXiv:1802.03162*, 2018.
- [9] F. Feng et al., "The application of a novel neural network in the detection of phishing websites," *J. Ambient Intell. Humaniz. Comput.*, 2020.
- [10] R. S. Rao and A. R. Pais, "Detection of phishing websites using an efficient feature-based ML framework," *Neural Comput. Appl.*, vol. 31, no. 8, pp. 3851–3873, 2019.
- [11] T. Tiwari, "Phishing Site URLs Dataset," Kaggle. [Online]. Available: <https://www.kaggle.com/datasets/taruntiwarihp/phishing-site-urls>
- [12] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.
- [13] APWG, "Phishing Activity Trends Reports," Anti-Phishing Working Group, 2024. [Online]. Available: <https://apwg.org>

APPENDICES

Appendix A: System Architecture Summary

The system follows a five-stage architecture: (1) URL input via Flask web form, (2) preprocessing via regex to strip protocol prefixes, (3) TF-IDF vectorization using the pre-fitted vectorizer, (4) classification using either the LR or MNB model, and (5) result display with probability score.

Appendix B: Key Performance Indicators

Logistic Regression: 96.4% accuracy | 95.8% precision | 94.2% recall | 95.0% F1 | AUC 0.984 | < 1 ms inference

Multinomial Naive Bayes: 94.1% accuracy | 93.5% precision | 91.8% recall | 92.6% F1 | AUC 0.971 | < 1 ms inference

Appendix C: Technologies Used

- Python 3.10 Core programming language
- scikit-learn 1.3 ML pipeline and classifiers
- Flask 3.0 Web application framework
- TF-IDF Vectoriser Character n-gram feature extraction (char_wb, 3–5g, 500K features)
- pickle Model serialisation
- Kaggle Dataset 549,347 labelled URLs from PhishTank, DMOZ, OpenPhish, ISCX-URL 2016